

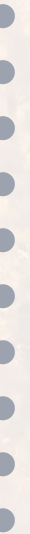
Control Statements

Mastering Break, Continue, and Exit in C Programming

 break

 continue

 exit





The Break Statement



Sometimes, it is required to **jump out of a loop** irrespective of conditional test value. Break statement is used inside any loop to allow control jump to immediate statement following the loop.

Syntax:

```
break;
```



Nested Loops

When nested loops are used, break jumps control from the loop where it has been used.



Compatibility

Break statement can be used inside any loop: while, do-while, for, and also in switch statement.





Break Statement Example

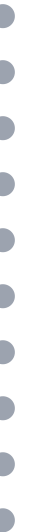
```
#include<stdio.h>
#include<conio.h>
main() {
    int i;
    clrscr();
    for(i = 1; i <= 20; i++) {
        if(i > 5)
            break;
        printf("%d", i); // 5 times only
    }
    printf("\nOut of loop");
    getch();
}
```

Output:

```
1 2 3 4 5
Out of loop
```



Explanation: The loop starts with $i = 1$ and continues until $i > 5$, when break statement is executed, exiting the loop early.



The Continue Statement

 Sometimes, it is required to **skip a part of a body of loop** under specific conditions. C supports 'continue' statement to overcome this anomaly.

Syntax:

```
continue;
```

◆ Working Structure

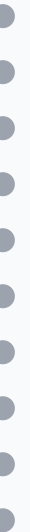
Similar to break statement but difference is that it cannot terminate loop.

◆ Function

Causes loop to be continued with next iteration after skipping statements in between.



Key Point: Continue statement simply skips statements and continues next iteration.





Continue Statement Example

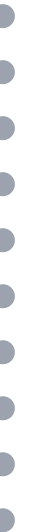
```
#include<stdio.h>
#include<conio.h>
main() {
    int i;
    clrscr();
    for(i = 1; i <= 10; i++) {
        if(i >= 6 && i <= 8)
            continue;
        printf("\t%d", i); // 6 to 8 is omitted
    }
    getch();
}
```

Output:

1 2 3 4 5 9 10



Explanation: When i is between 6 and 8, continue statement skips the printf and moves to next iteration.



The Exit Function

 The **exit()** function is used to terminate execution of 'C' program. It is a standard library function and uses header file `stdlib.h`.

General Form:

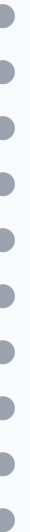
```
exit(int status);
```

◆ Status Parameter

Value returned to operating system after termination of the program.

◆ Status Values

Zero "0" indicates normal termination, while "1" (Non-Zero) indicates different types of errors.





Exit Function Example

```
#include<stdio.h>
#include<conio.h>
main() {
    int i;
    clrscr();
    for(i = 1; ; i++) {
        if(i > 5)
            exit(0);
        printf("%d", i); // 5 times only
    }
    printf("\nOut of loop"); // control will not reach here
    getch();
}
```

Output:

1 2 3 4 5



Explanation: When $i > 5$, `exit(0)` terminates the entire program, so the "Out of loop" message is never printed.



Break vs. Exit

Break Statement

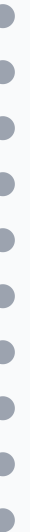
- ✓ Terminates execution of loop only
- ✓ Program continues after loop
- ✓ Used within loops and switch

Exit Function

- ✓ Terminates execution of program itself
- ✓ Returns status to operating system
- ✓ Requires `stdlib.h` header file



Key Difference: Break terminates the loop while exit terminates the entire program.





Conclusion

A program is usually not limited to a linear sequence of instructions. During its process it may require to repeat execution of a part of code more than once depending upon requirements or take decisions.



Break

Leave a loop even if condition for its end is not fulfilled



Continue

Skip rest of loop in present iteration and jump to next



Exit

Terminate the entire program and return status to OS



Using goto statement, we can make an absolute jump to another point in program. Use this feature carefully since its execution ignores any type of nesting limitation.